

第二章

并发进程

主要内容

- 4.1 UNIX的进程调度状态
- 4.2 UNIX的进程切换调度
- 4.3 UNIX的进程创建与终止
- 4.4 UNIX的进程图像交换
- 4.5 进程通信**



并发执行的进程之间相互影响相互制约



如何确保进程之间的相互影响不会出错?

进程互斥



进程互斥



实例



Process IN:

...
count++;
...



```
mov ax, [ ... ];  
inc ax;  
mov [ ... ], ax;
```

Process OUT:

...
count--;
...



```
mov ax, [ ... ];  
dec ax;  
mov [ ... ], ax;
```

```
mov ax, [ ... ];  
inc ax;  
mov [ ... ] ax;  
mov ax, [ ... ];  
dec ax;  
mov [ ... ] ax;
```

ax count

100 ← **100**

101

101 → 101

101 ← 101

100

100 → **100**

```
mov ax, [ ... ];  
mov ax, [ ... ];  
dec ax;  
mov [ ... ] ax;  
inc ax;  
mov [ ... ] ax;
```

ax count

100 ← **100**

100 ← 100

99

99 → 99

101

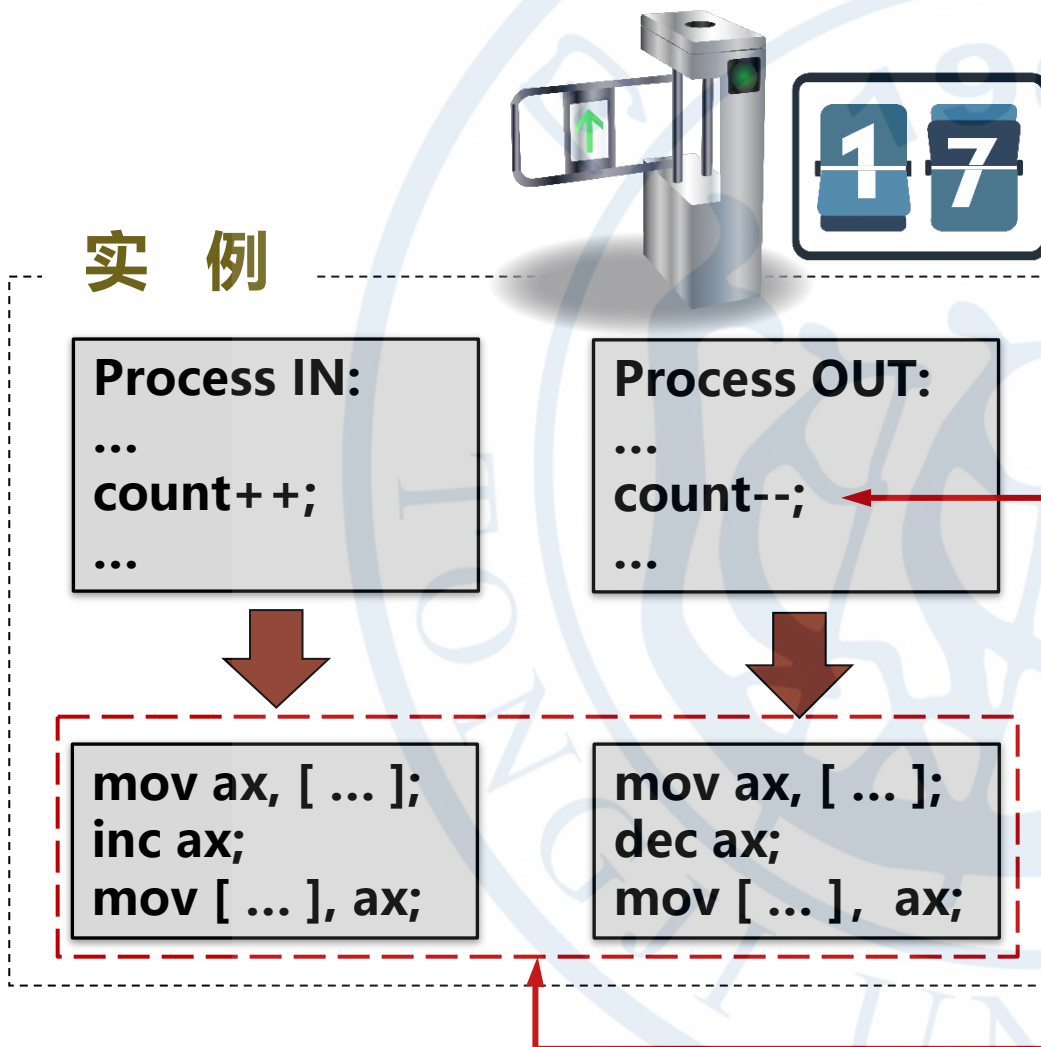
101 → **101**



进程互斥



实例



临界资源：同时只允许一个进程使用的软件和硬件资源。

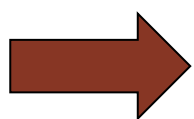
临界区：访问临界资源的程序段。
相关临界区：访问同一临界资源的临界区。

竞争条件：多个进程同时读写共享的数据，而最终的结果取决于进程执行的时序。

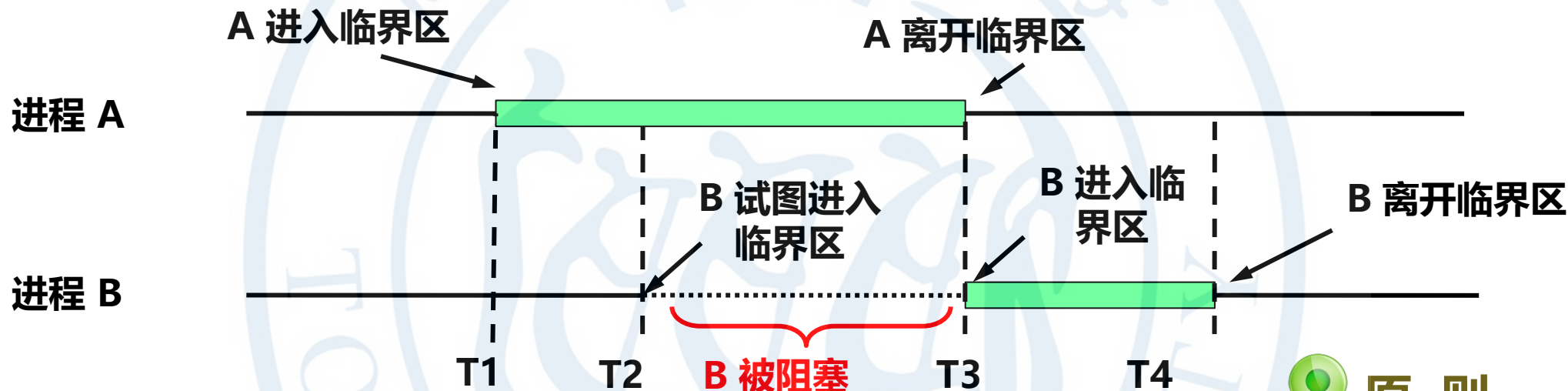
包含竞争条件的程序是错误的（与时序相关的错误）。



进程互斥



同时使用同一临界资源的进程应当**互斥**：保证在任何时刻，最多只有一个进程运行在临界区。



入口；

//检测临界资源是否空闲？

//如果空闲，则进入临界区，设置忙标志；

//否则，忙等或稍后再试；

临界区；

出口；

// 清除临界资源的忙标志。



原则

- 空闲让进
- 忙则等待
- 有限等待
- 让权等待



进程互斥



1. **开关中断**：在单处理机系统中，借用中央处理机中的硬件中断开关位作为临界区的锁。

关中断

//中断关闭，不会发生进程切换
//当前进程不被打断执行临界区程序

critical section;

开中断

// 离开临界区后开中断

适合操作系统内核的进程互斥

优点：简单，可靠。

不足：1) 单处理器

2) 临界区中不能包含可能进入阻塞状态的因素

3) 临界区比较长时，.....

中断只关闭了抢占调度，
没有关闭非抢占调度



进程互斥



2. **锁变量**: 为每个临界资源设置一个锁变量lock, lock=0表示锁开着; lock=1表示锁关闭 (初始为0)。

```
while ( lock == 1)
{ };
```

```
lock = 1;
```

```
临界区;
```

```
lock = 0;
```

//试图进入临界区前先检测锁的状态;
//若锁为关闭状态, 则等待。
//若锁为打开状态, 则开锁后进入临界区

如果这里发生了进程调度?

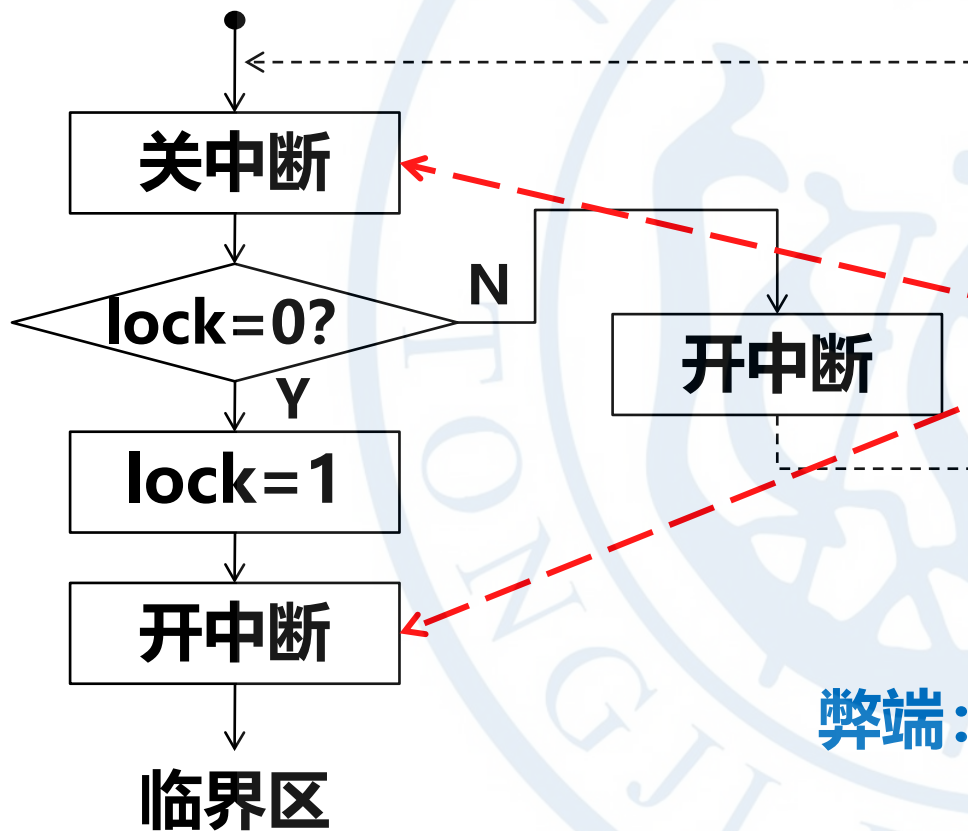
//退出临界区后, 开锁

有可能有多个进程同时进入临界区

解决问题的关键: 锁变量的检查和置1应同时完成 (关锁操作的完整性)



2. **锁变量**：为每个临界资源设置一个锁变量lock，lock=0表示锁开着；lock=1表示锁关闭（初始为0）。

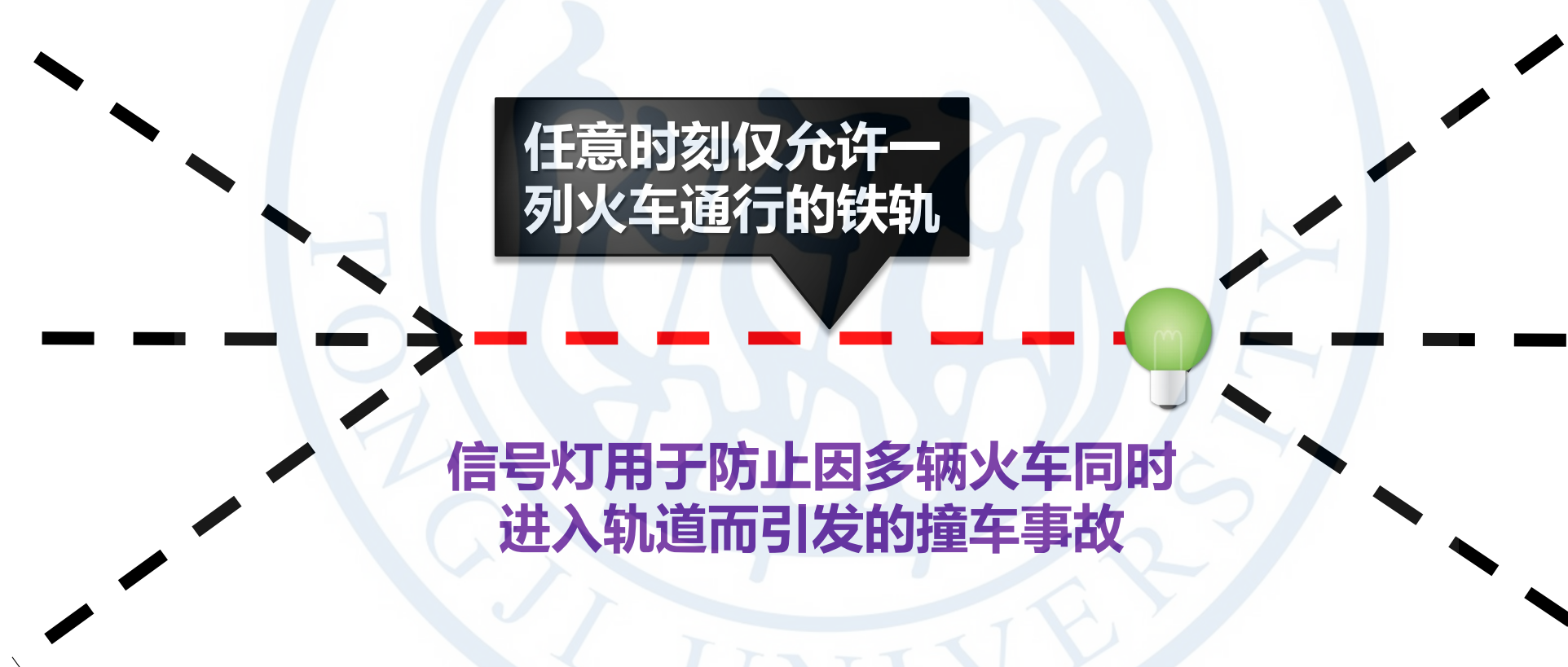


利用开关中断来保证关锁操作的完整性，并不是实施关锁操作本身。

弊端：忙等 浪费了宝贵的CPU时间

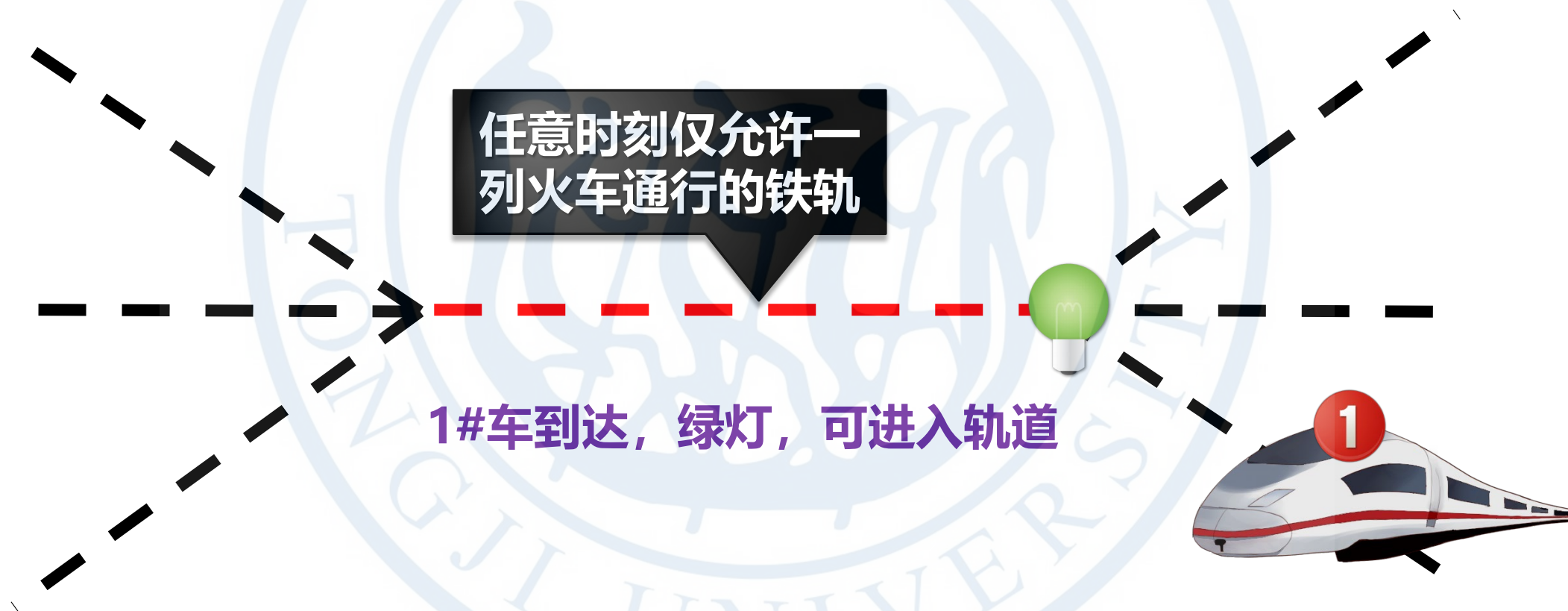


E. W. Dijkstra受铁路信号灯的启发，发明信号量机制，用来解决操作系统进程互斥问题





E. W. Dijkstra受铁路信号灯的启发，发明信号量机制，用来解决操作系统进程互斥问题





E. W. Dijkstra受铁路信号灯的启发，发明信号量机制，用来解决操作系统进程互斥问题



1#车进入轨道后，信号灯变红，
阻止其他车辆进入轨道



进程互斥



利用信号量实现进程互斥

E. W. Dijkstra受铁路信号灯的启发，发明信号量机制，用来解决操作系统进程互斥问题





进程互斥



利用信号量实现进程互斥

E. W. Dijkstra受铁路信号灯的启发，发明信号量机制，用来解决操作系统进程互斥问题





利用信号量实现进程互斥

E. W. Dijkstra受铁路信号灯的启发，发明信号量机制，用来解决操作系统进程互斥问题





E. W. Dijkstra受铁路信号灯的启发，发明信号量机制，用来解决操作系统进程互斥问题

需要一种机制来描述系统状态



信号灯S

- **红** 轨道被占用
- **绿** 轨道空闲

FIFO的列车队列

- 等待通过的火车依次排在队尾
- 信号灯由红转绿时，队首火车进入轨道



E. W. Dijkstra受铁路信号灯的启发，发明信号量机制，用来解决操作系统进程互斥问题

火车过轨道问题

vs.

进程互斥问题

- | | | |
|-------------|--------|----------------|
| ➤ 轨道 | -----➤ | ➤ 一个互斥访问的临界资源 |
| ➤ 火车 | -----➤ | ➤ 进程 |
| ➤ 火车过轨道的过程 | -----➤ | ➤ 进程执行临界区代码的过程 |
| ➤ 描述轨道的使用状态 | -----➤ | ➤ 描述资源使用状态的信号量 |

- | | | |
|-------------|--------|--|
| ➤ 灯的颜色（红/绿） | -----➤ | int value ; // value = 1 绿灯 ;
value = 0 红灯 ; |
| ➤ FIFO火车队列 | -----➤ | struct sem_queue * head ; //所有等待使用该资源的进程排成的队列 |

初值为1的信号量称为互斥信号量

```
struct semaphore
{
    int value;   // value = 1   绿灯;  
                                          value = 0   红灯;  
    struct sem_queue *head;   //所有等待使用该资源的进程排成的队列
}
```



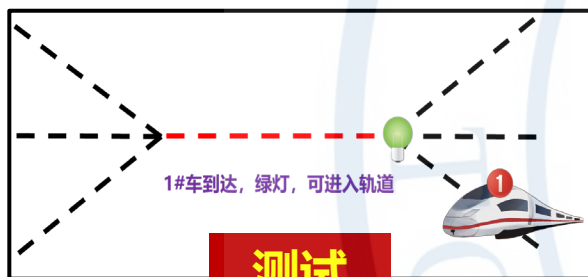
E. W. Dijkstra受铁路信号灯的启发，发明信号量机制，用来解决操作系统进程互斥问题

火车过轨道问题

vs.

进程互斥问题

➤ 进轨道前



测试



进入，封锁

➤ 出轨道时



离开，释放



放行

➤ 使用临界资源前

```
void P(s)
{
    s.value = s.value-1;
    if (s.value < 0) sleep();
}
```

➤ 释放临界资源时

```
void V(s)
{
    s.value = s.value+1;
    if (s.value < ?=0) wakeup();
}
```



进程互斥



利用信号量实现进程互斥

更一般的信号量

```
struct semaphore
{
    int value; //信号量的值
    struct sem_queue *head; //等待使用该信
    号量的进程排成的队列
};
```



信号量的初始化

- (1) 为value设置初值
- (2) sem_queue队列指针设置为空

1. 初值:
初始条件下，可使用的资源数

$s.value = 3$

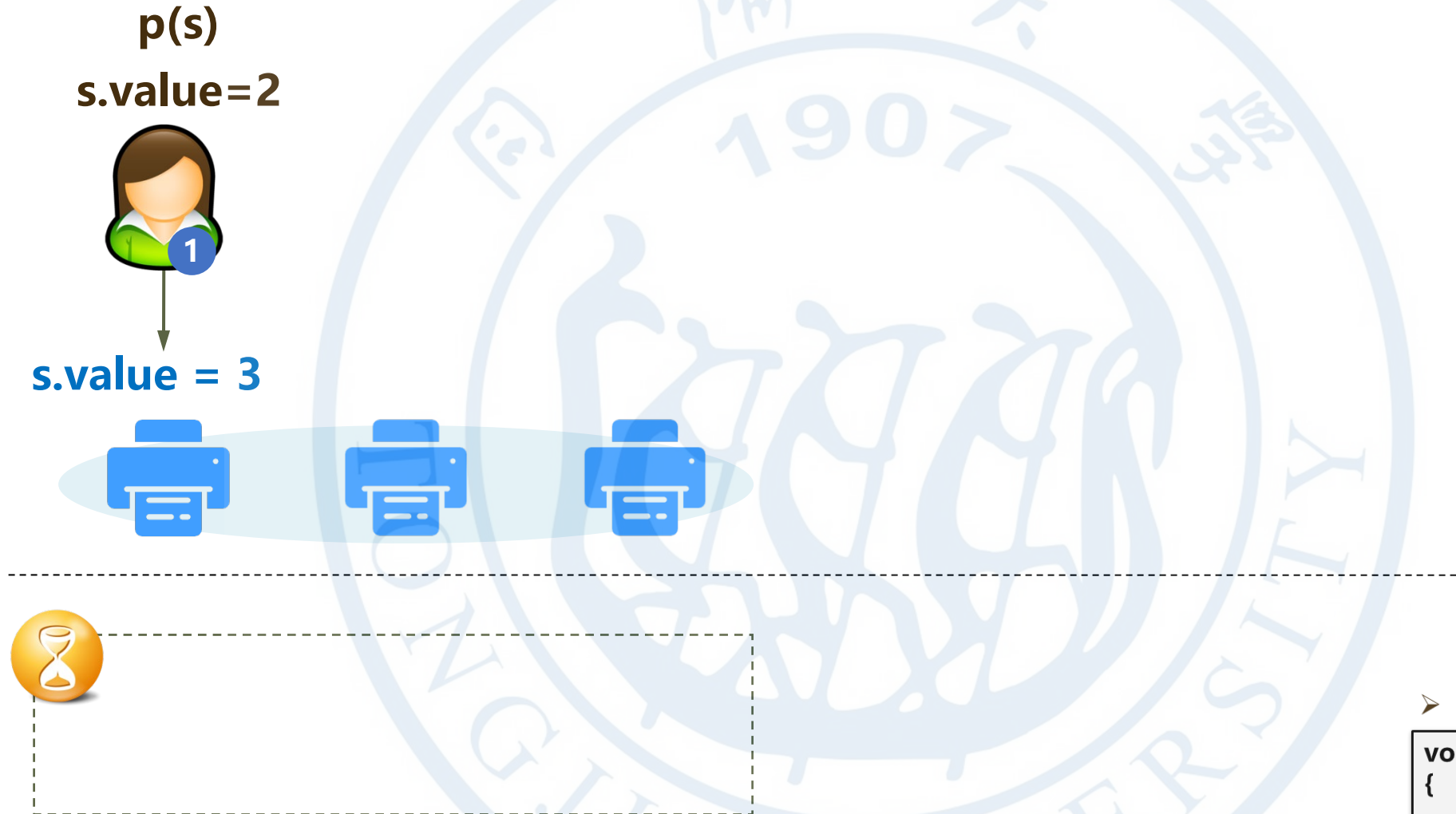




进程互斥



利用信号量实现进程互斥



➤ 使用临界资源前

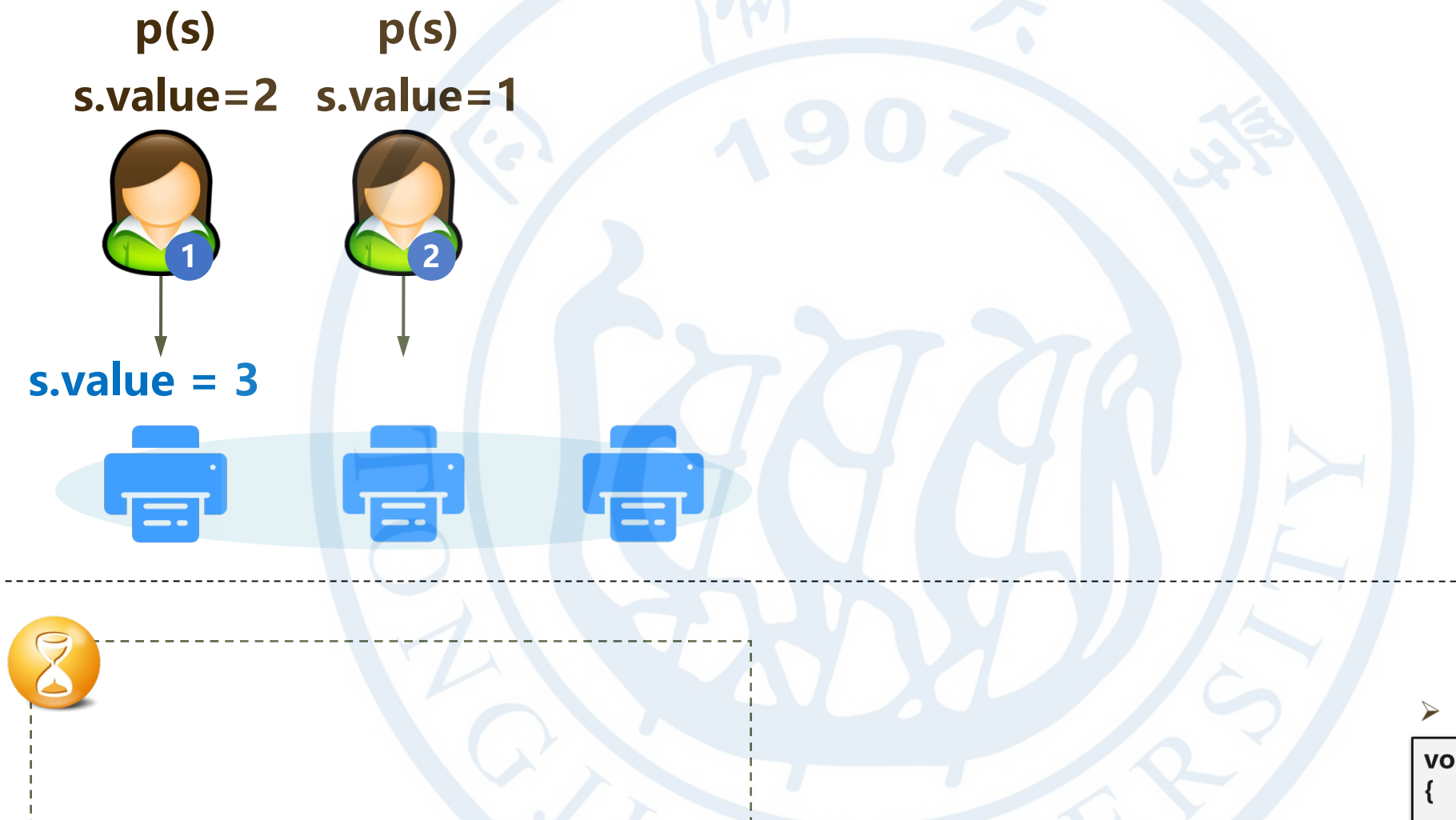
```
void P(s)
{
    s.value = s.value-1;
    if (s.value < 0) sleep();
}
```



进程互斥



利用信号量实现进程互斥



➤ 使用临界资源前

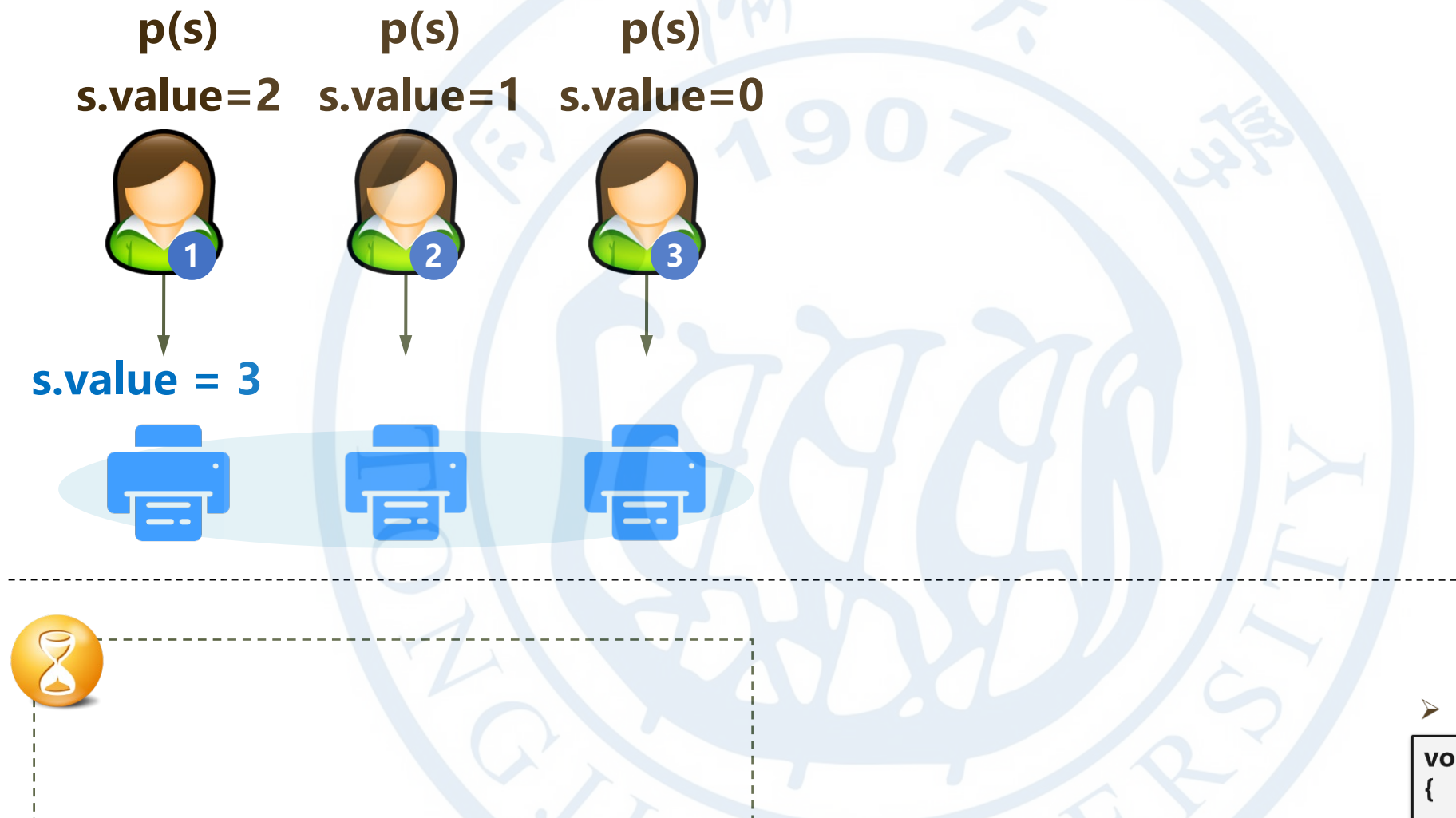
```
void P(s)
{
    s.value = s.value-1;
    if (s.value < 0) sleep();
}
```



进程互斥



利用信号量实现进程互斥



➤ 使用临界资源前

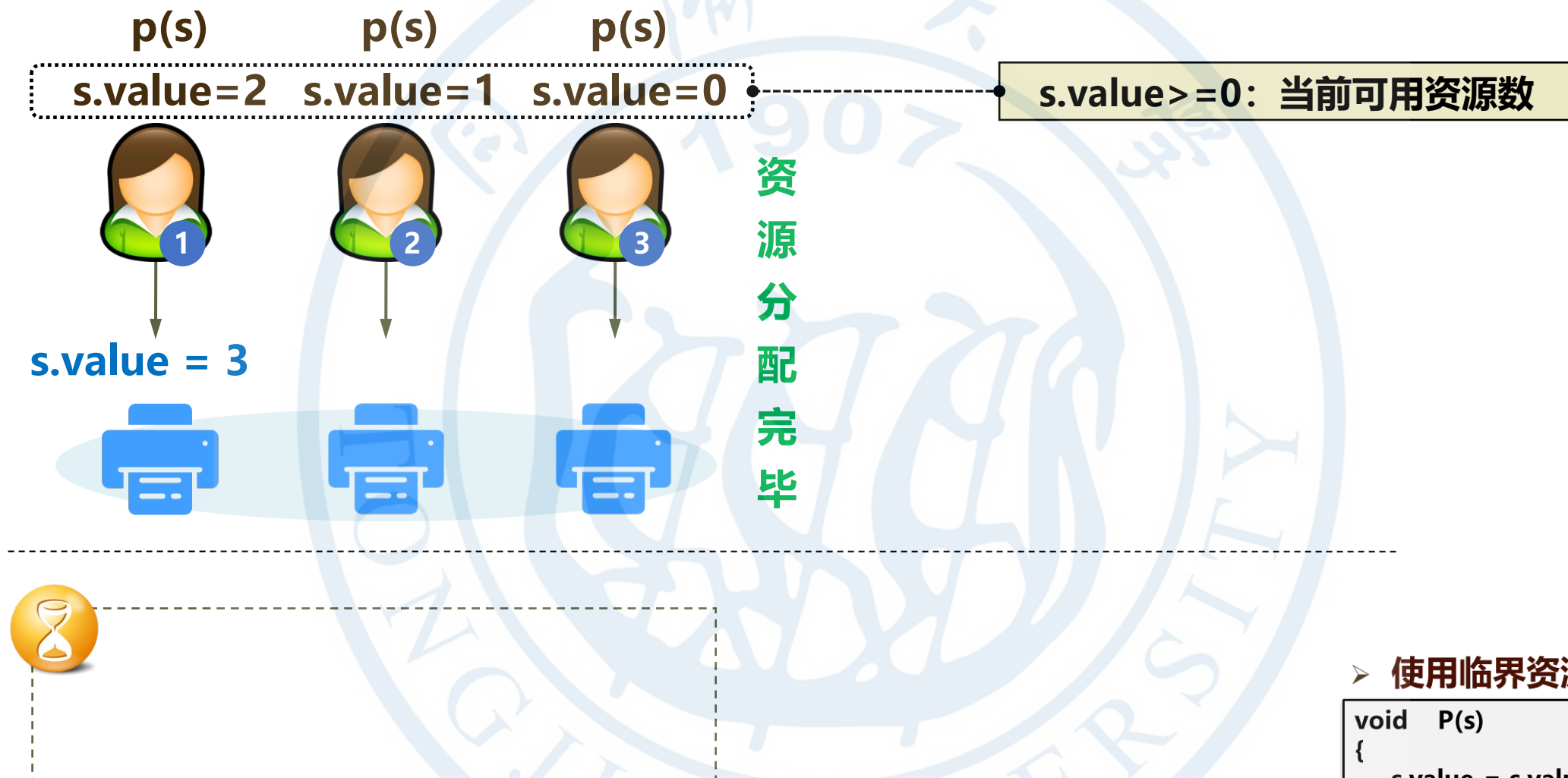
```
void P(s)
{
    s.value = s.value-1;
    if (s.value < 0) sleep();
}
```



进程互斥



利用信号量实现进程互斥



使用临界资源前

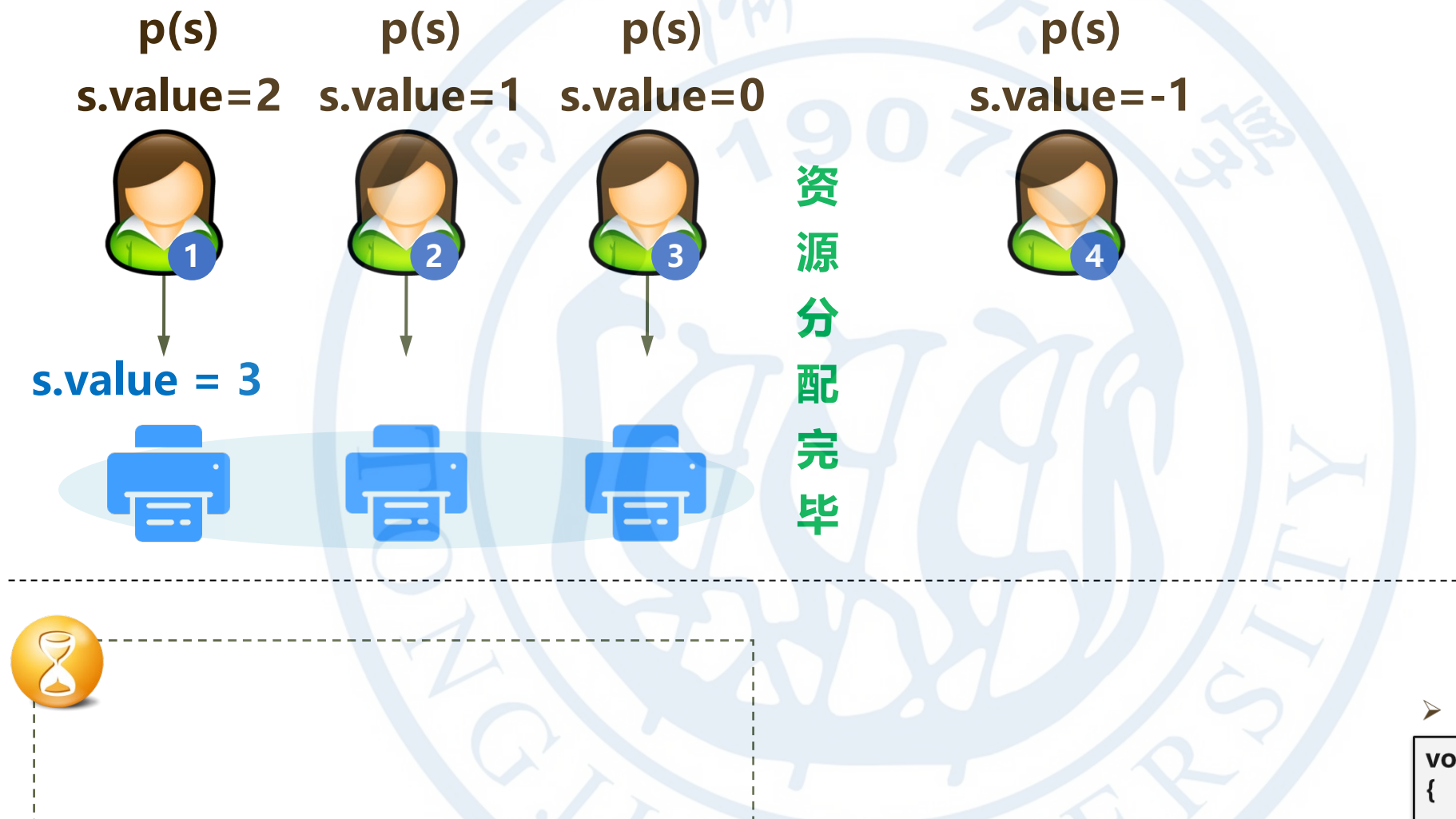
```
void P(s)
{
    s.value = s.value - 1;
    if (s.value < 0) sleep();
}
```



进程互斥



利用信号量实现进程互斥



➤ 使用临界资源前

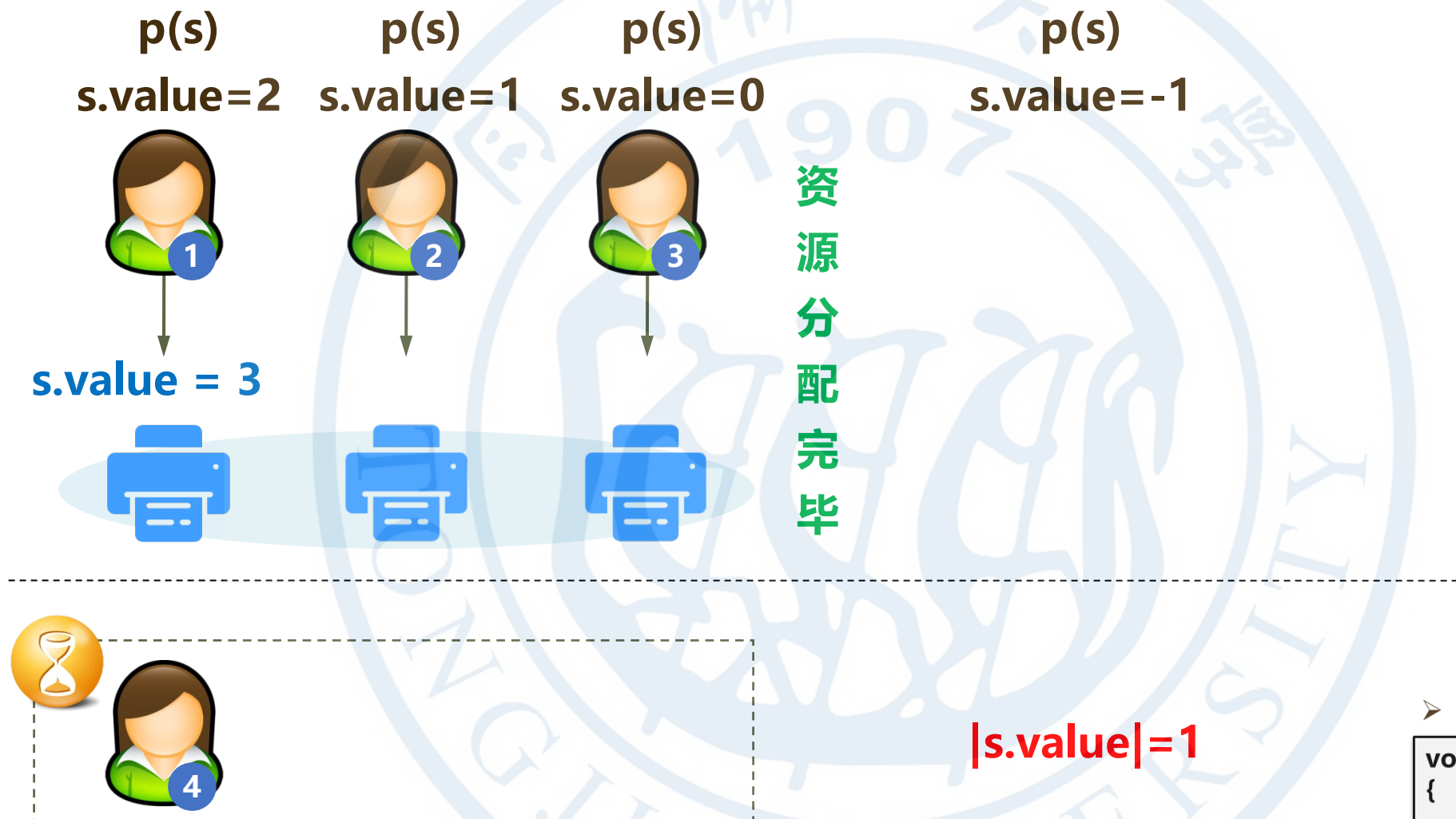
```
void P(s)
{
    s.value = s.value-1;
    if (s.value < 0) sleep();
}
```



进程互斥



利用信号量实现进程互斥



➤ 使用临界资源前

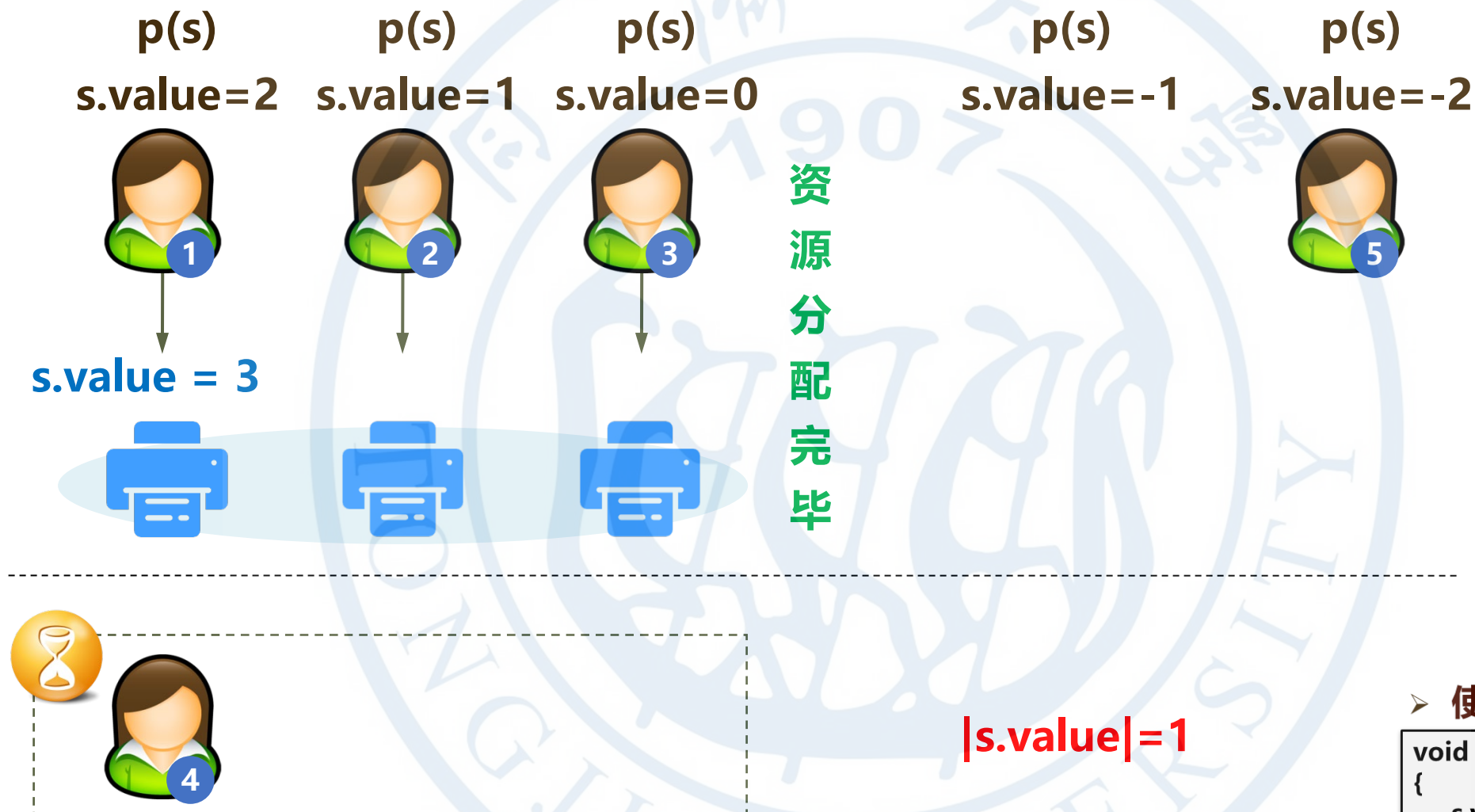
```
void P(s)
{
    s.value = s.value - 1;
    if (s.value < 0) sleep();
}
```



进程互斥



利用信号量实现进程互斥



➤ 使用临界资源前

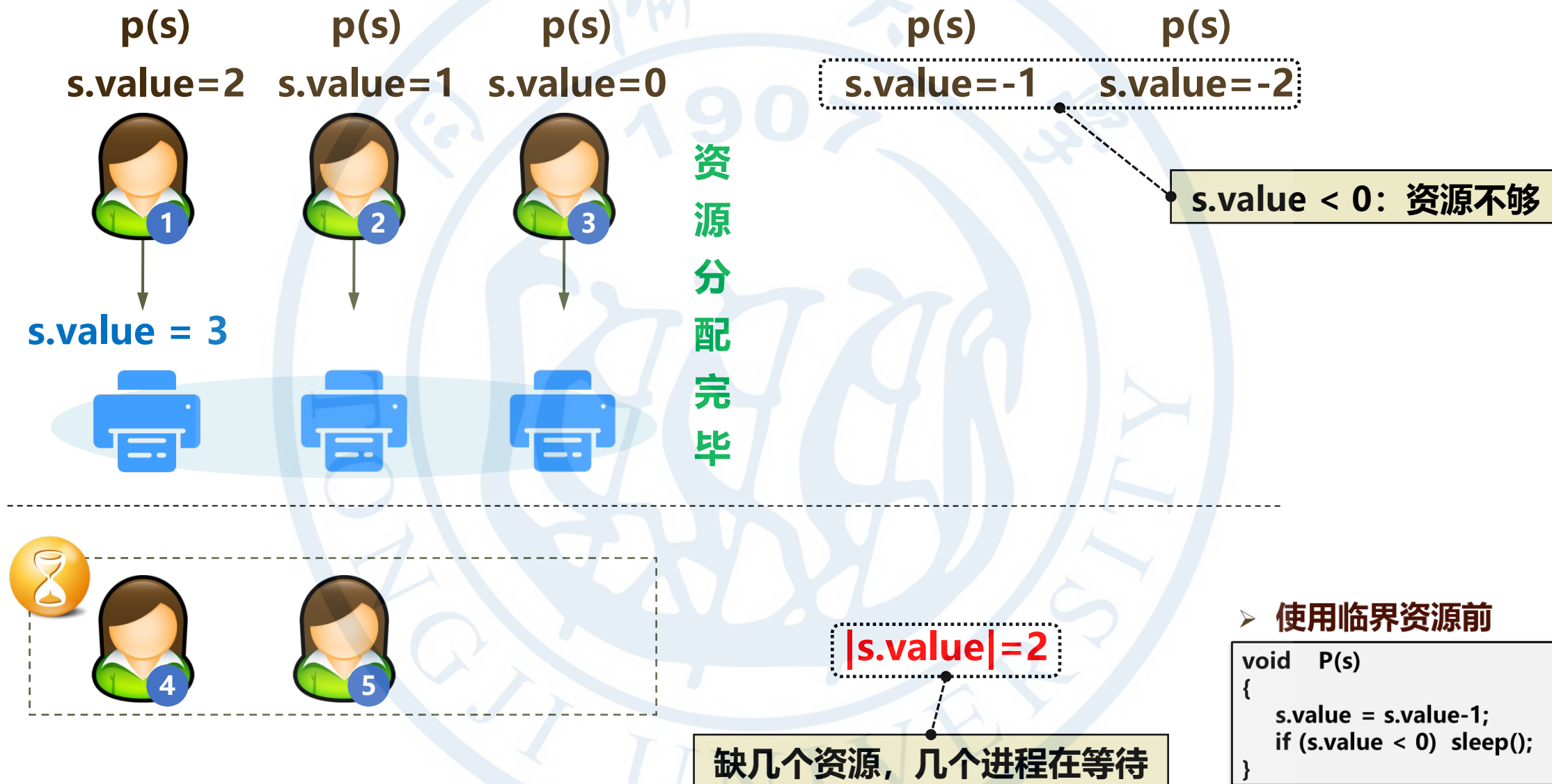
```
void P(s)
{
    s.value = s.value-1;
    if (s.value < 0) sleep();
}
```



进程互斥



利用信号量实现进程互斥

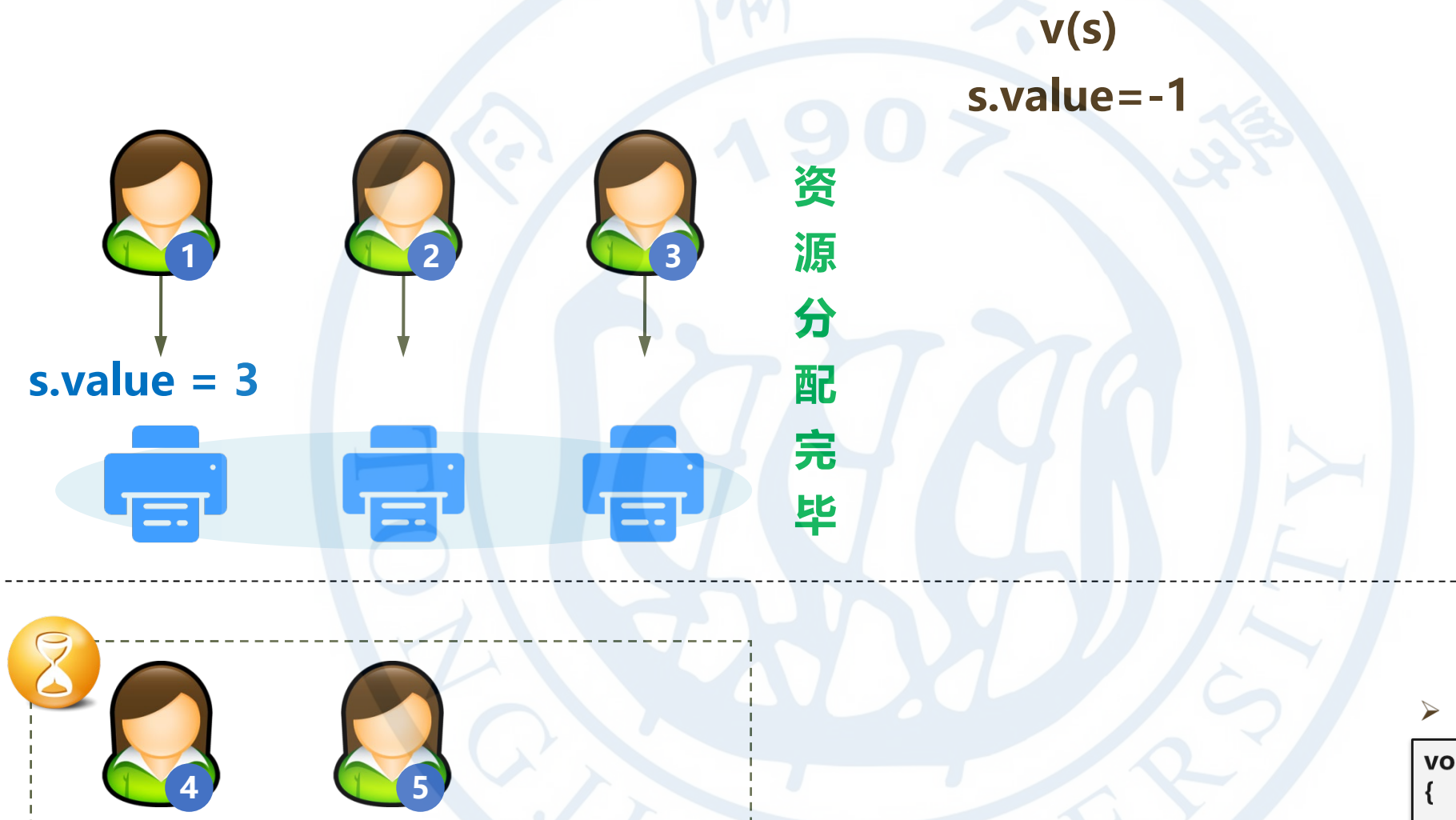




进程互斥



利用信号量实现进程互斥



➤ 释放临界资源时

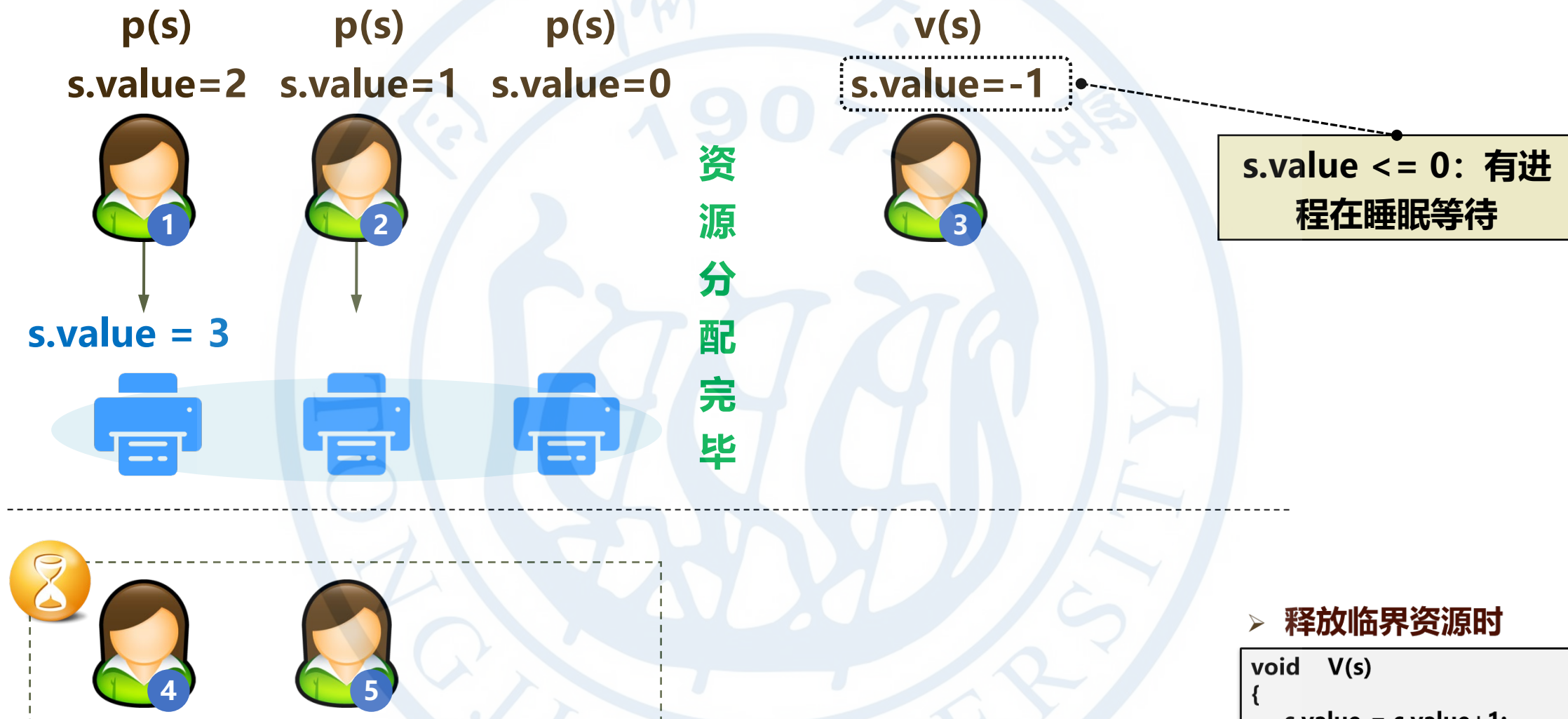
```
void V(s)
{
    s.value = s.value + 1;
    if (s.value < ? 0) wakeup();
}
```



进程互斥



利用信号量实现进程互斥



➤ 释放临界资源时

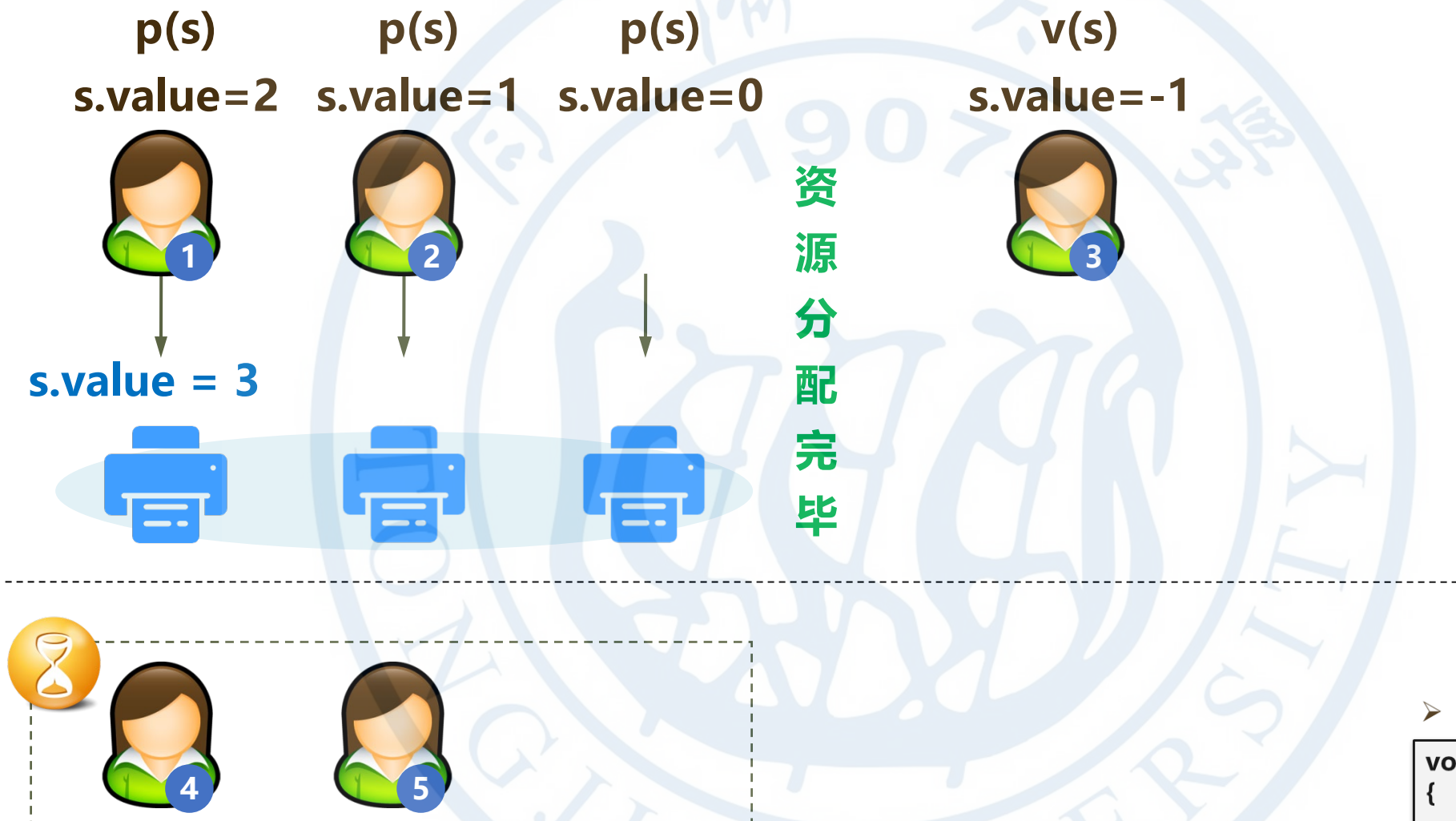
```
void V(s)
{
    s.value = s.value + 1;
    if (s.value < ? 0) wakeup();
}
```



进程互斥



利用信号量实现进程互斥



➤ 释放临界资源时

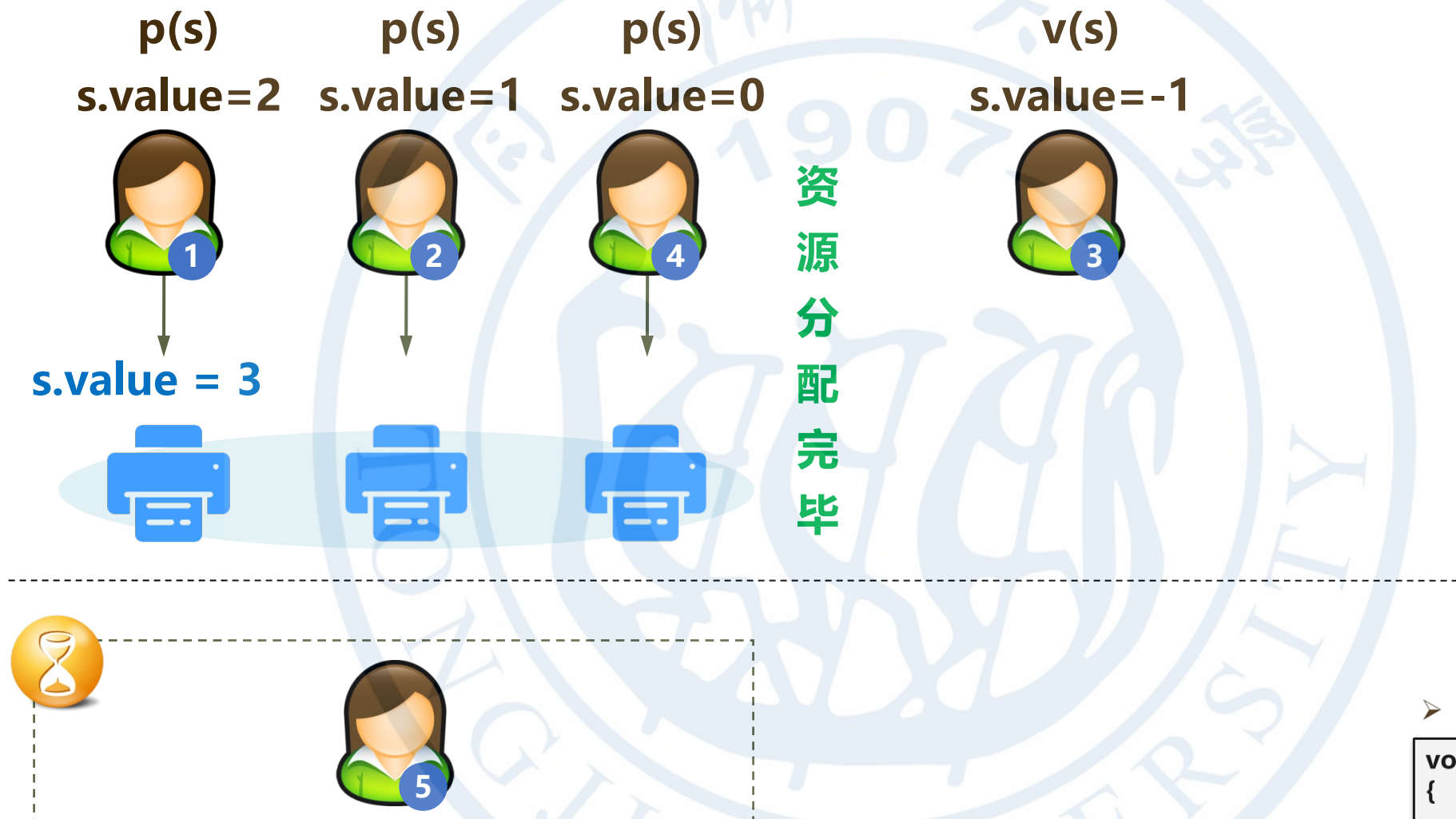
```
void V(s)
{
    s.value = s.value + 1;
    if (s.value < ? 0) wakeup();
}
```



进程互斥



利用信号量实现进程互斥



➤ 释放临界资源时

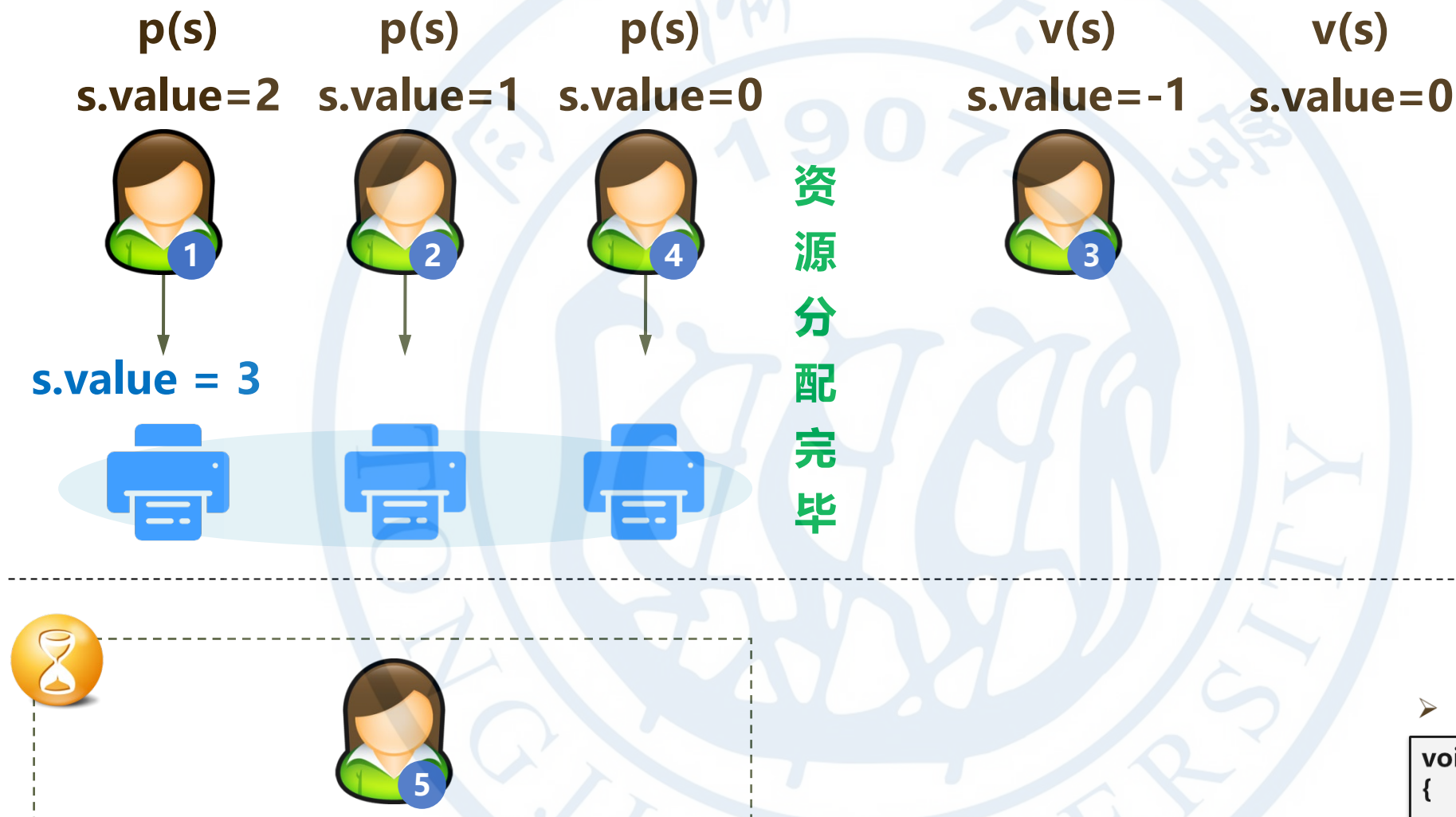
```
void V(s)
{
    s.value = s.value+1;
    if (s.value < ?=0) wakeup();
}
```



进程互斥



利用信号量实现进程互斥



➤ 释放临界资源时

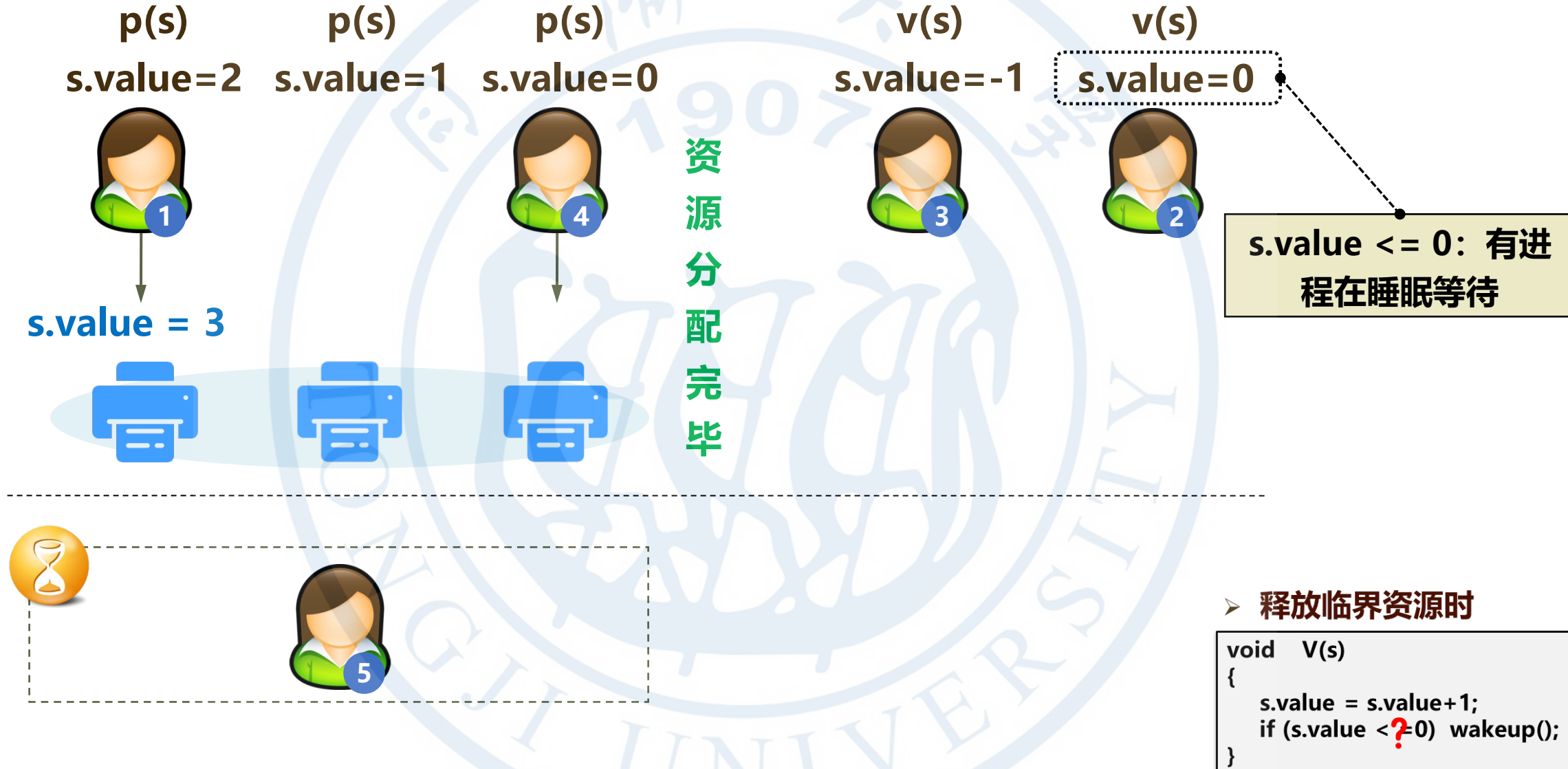
```
void V(s)
{
    s.value = s.value + 1;
    if (s.value < ?= 0) wakeup();
}
```



进程互斥



利用信号量实现进程互斥

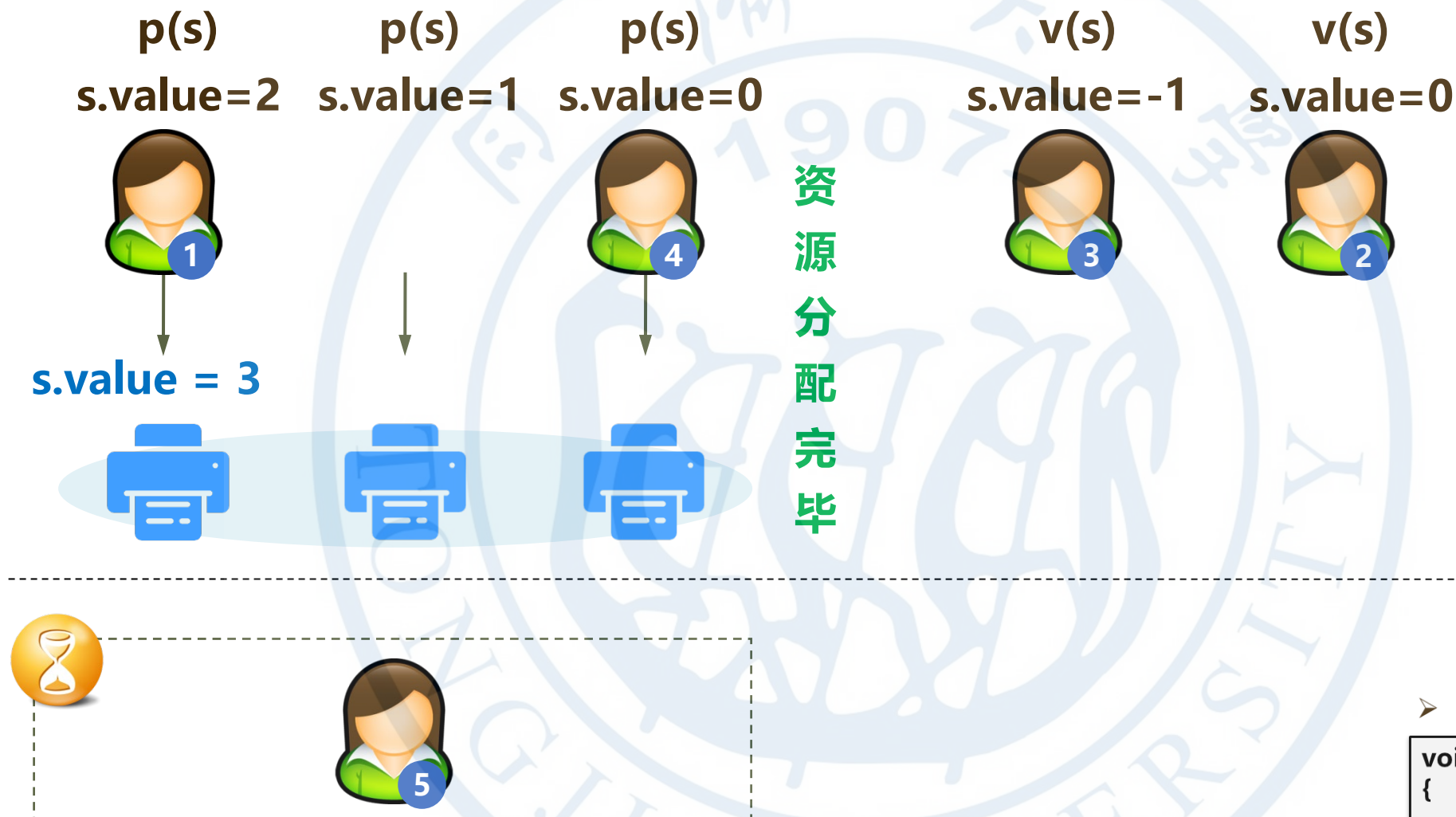




进程互斥



利用信号量实现进程互斥



➤ 释放临界资源时

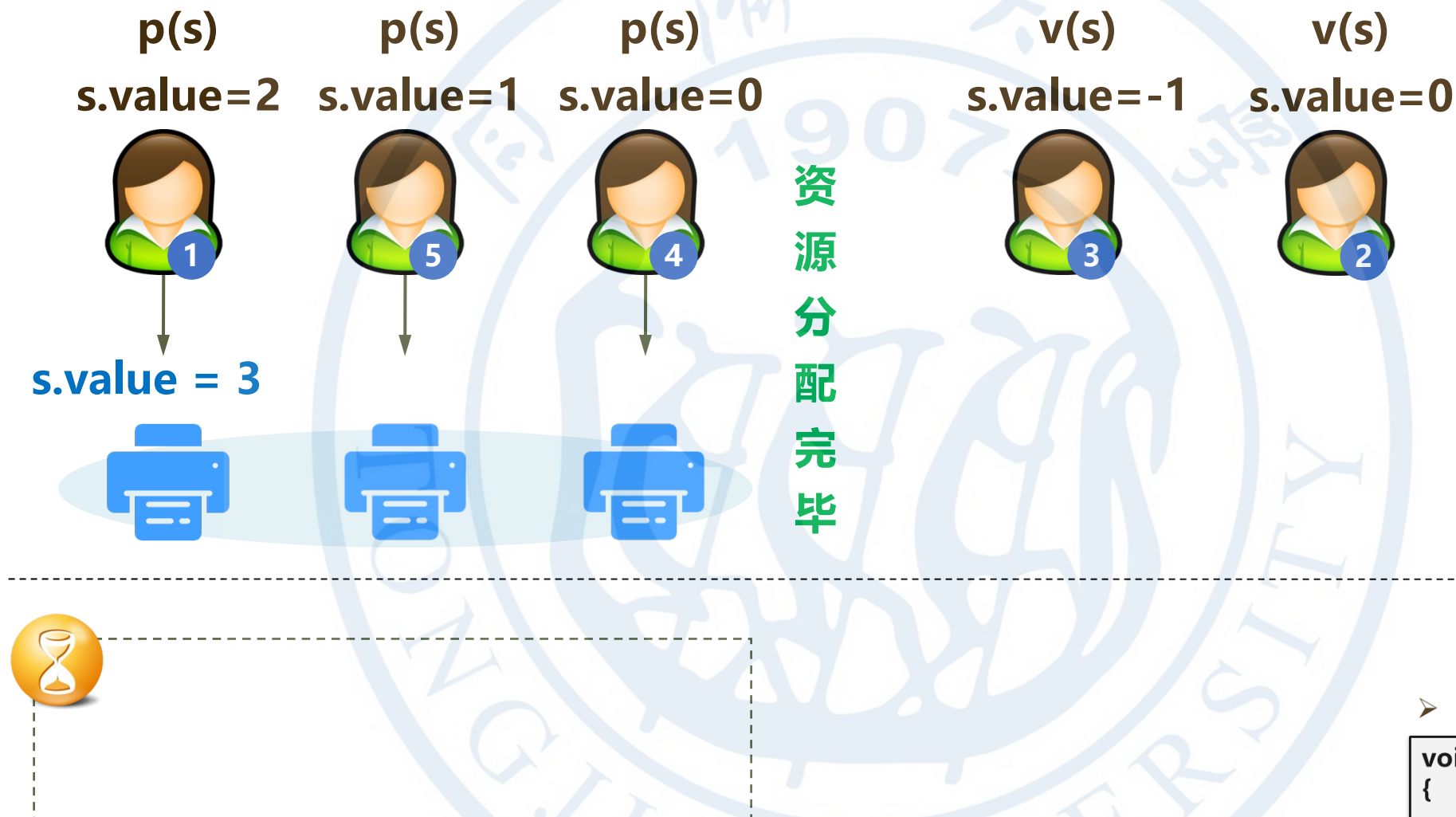
```
void V(s)
{
    s.value = s.value+1;
    if (s.value < ?=0) wakeup();
}
```



进程互斥



利用信号量实现进程互斥



➤ 释放临界资源时

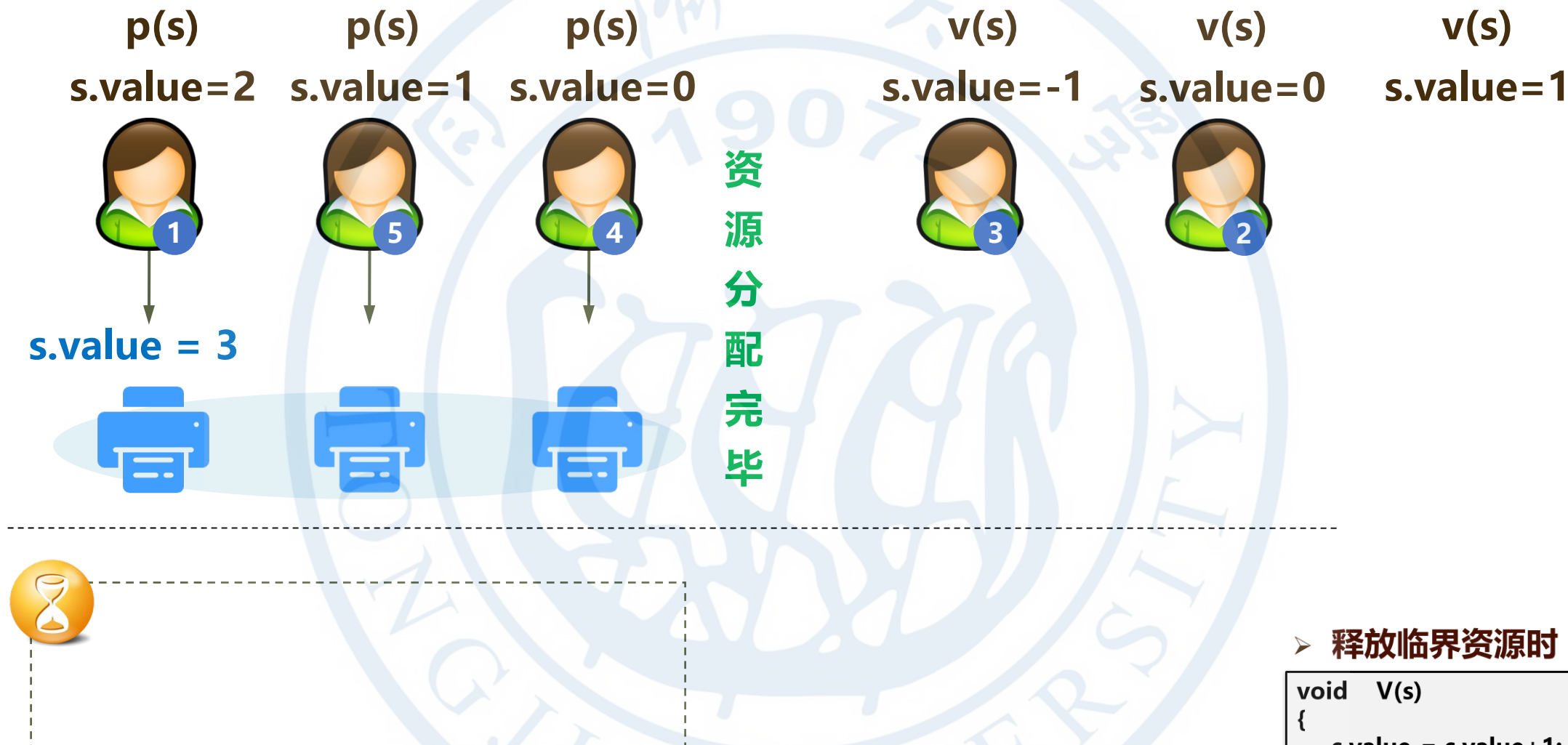
```
void V(s)
{
    s.value = s.value + 1;
    if (s.value < ?= 0) wakeup();
}
```



进程互斥



利用信号量实现进程互斥



➤ 释放临界资源时

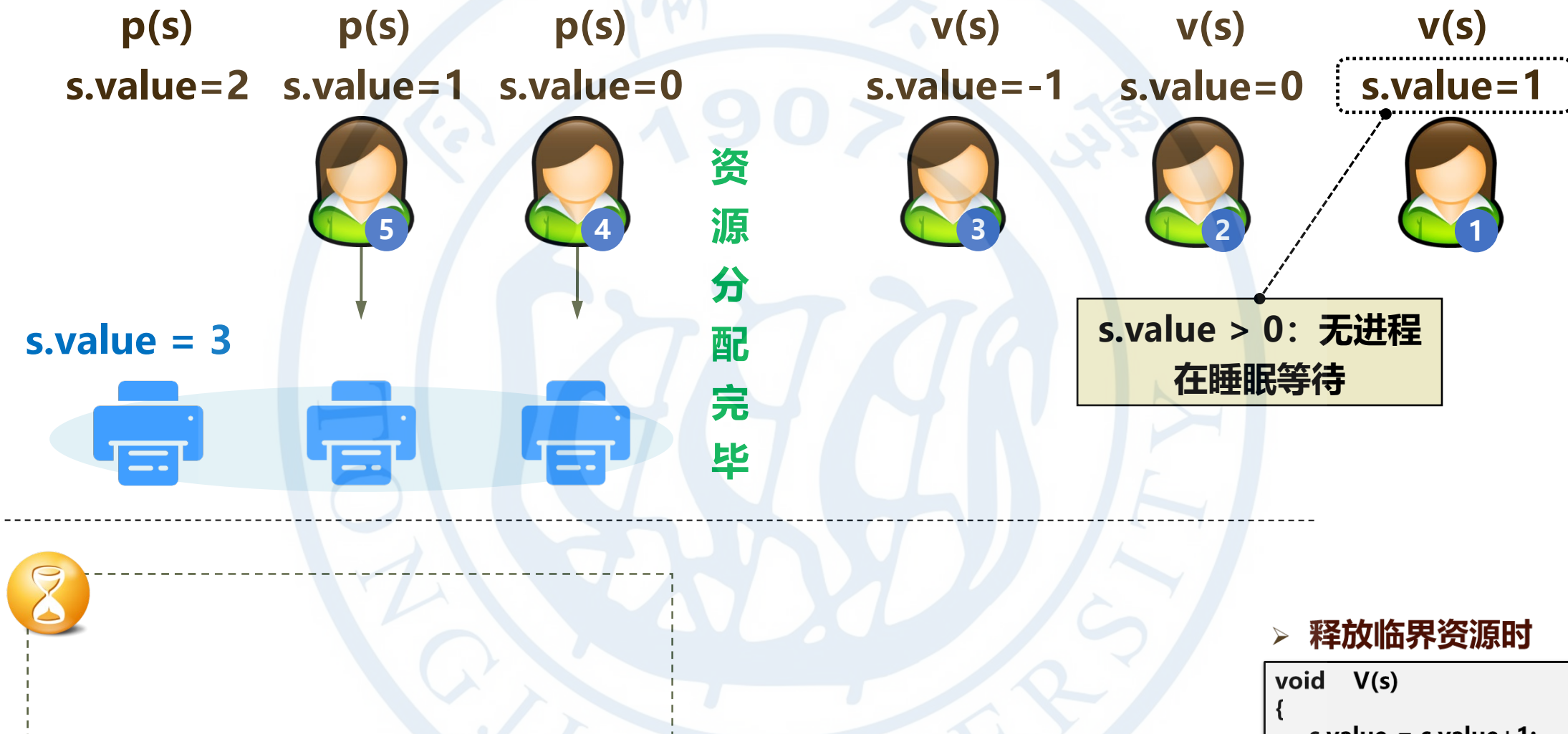
```
void V(s)
{
    s.value = s.value+1;
    if (s.value < ?=0) wakeup();
}
```



进程互斥



利用信号量实现进程互斥



➤ 释放临界资源时

```
void V(s)
{
    s.value = s.value + 1;
    if (s.value < ? 0) wakeup();
}
```



进程互斥



更一般的信号量

```
struct semaphore
{
    int value; //信号量的值
    struct sem_queue *head; //等待使用该信
    号量的进程排成的队列
};
```



信号量的初始化

- (1) 为value设置初值
- (2) sem_queue队列指针设置为空

1. 初值:
初始条件下，可使用的资源数

s.value = 3





更一般的信号量

```
void P(s)
{
    s.value = s.value-1; //用减去1的方式来申请资源
    if (s.value < 0)      //如果该类资源已分配完毕，则申请资源失败
        sleep();         //本进程进入该信号量的等待队列中
}

void V(s)
{
    s.value = s.value+1; //用加上1的方式释放一个资源
    if (s.value <= 0)     //仍有进程在等待使用本信号量
        wakeup();        //从本信号量的等待队列中唤醒一个进程
}
```

由操作系统内核保证P，V操作的原子性（内核原语）

必须成对出现



进程互斥



利用信号量实现进程互斥

互斥：任意时刻只能有一个进程使用该资源

Process IN:

```
...  
count++;  
...
```

Process OUT:

```
...  
count--;  
...
```

```
mov ax, count;  
inc ax;  
mov count ax;
```

```
mov ax, count;  
dec ax;  
mov count ax;
```

Process IN:

```
...  
P(mutex);  
count++;  
V(mutex);  
...
```

Process OUT:

```
...  
P(mutex);  
count--;  
V(mutex);  
...
```

互斥信号量:
semaphore mutex;
mutex.value = 1;

1. 每一组相关临界区
与一个信号量对应

2. 进入临界区：P操作
离开临界区：V操作

3. PV操作在每个相关临界区前后
对同一信号量成对使用



并发执行的进程之间相互影响相互制约



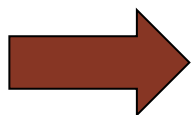
如何确保进程之间的相互影响不会出错？

进程互斥



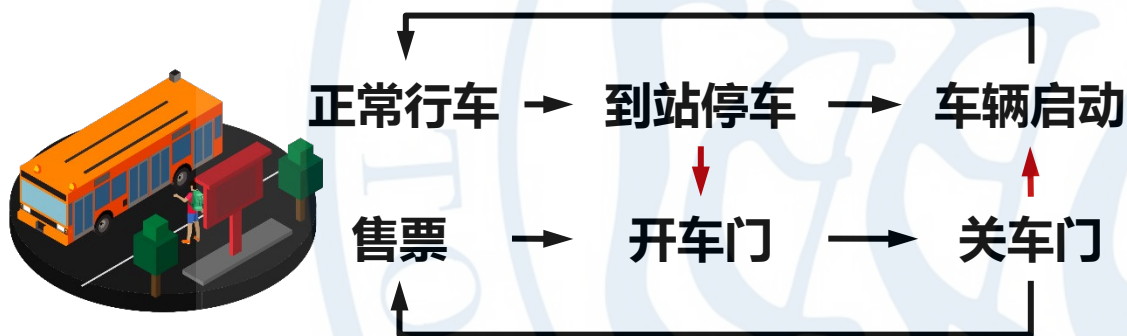
如何确保进程按合理的速度向前推进？

进程同步



协同工作的几个进程需要在某些确定的点上协调他们的工作。

实例



$func1(y)$

$func2(z)$

$$X = func1(y) * func2(z)$$

一个进程达到了一个确定的点（**同步点**）后，除非另一个进程已经完成了某些操作，否则就不得不停下来以等待这些操作执行结束。



同步：在同步点上等待“可以继续执行”的消息

Process pa:
...
L1: *P(proceed);*
...

Process pb:
...
L2: *V(proceed);*
...

同步信号量:
semaphore proceed;
proceed.value = 0;

需要一个“可以继续执行的消息”

发送一个“可以继续执行的消息”

1. 每一个消息与一个信号量对应

怎么理解：互斥
实质上是同步的一种特殊情况？

2. P操作：接收消息
V操作：发送消息

3. PV操作由不同的进程
实施，成对使用

同步：直接的相互制约关系

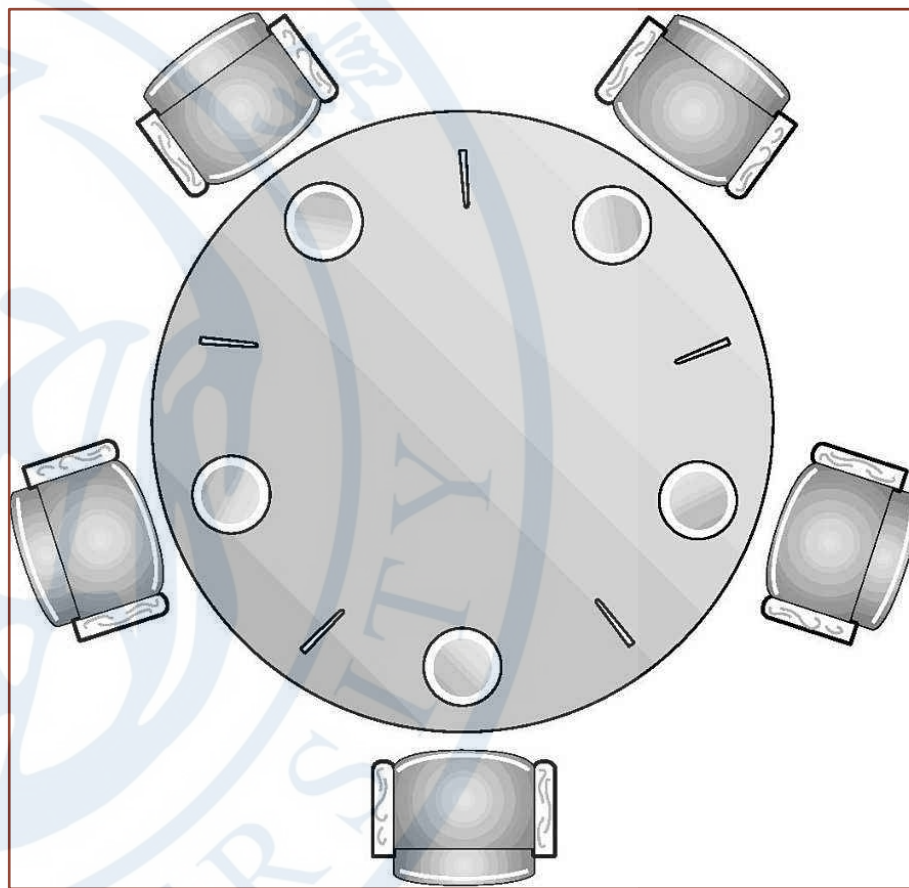
互斥：间接的相互制约关系



哲学家进餐问题



一张圆桌旁围坐着5名哲学家，每两个哲学家之间的桌上摆一根筷子，桌子的中间是一碗米饭。当哲学家饥饿的时候，只有同时拿到了两根筷子才可以开始进餐。如果筷子已在他人手上，则需等待。





哲学家进餐问题



```
void philosopher (int i)    //i 为执行此过程的哲学家号
{
    while (TRUE)
    {
        think();            //思考问题，直至有饥饿感

        eat();              //两把叉子都得到就开始进餐
    }
}
```



哲学家进餐问题



```
void philosopher (int i)    //i 为执行此过程的哲学家号
{
    while (TRUE)
    {
        think();            //思考问题，直至有饥饿感
        P(fork[i]);          //试图拿右侧叉子
        P(fork[(i+1)%N]);    //试图拿左侧叉子
        eat();               //两把叉子都得到就开始进餐
        V(fork[i]);          //放下右侧叉子
        V(fork[(i+1)%N]);    //放下左侧叉子
    }
}
```



哲学家进餐问题



```
void philosopher (int i)    //i 为执行此过程的哲学家号
{
    while (TRUE)
    {
        think();
        P(fork[i]);          //思考问题，直至有饥饿感
        P(fork[(i+1)%N]);    //试图拿右侧叉子
                               //试图拿左侧叉子
        eat();               //两把叉子都得到就开始进餐
        V(fork[i]);          //放下右侧叉子
        V(fork[(i+1)%N]);    //放下左侧叉子
    }
}
```

每位哲学家都取得右边一只筷子，并且不肯放弃左边的筷子。这种**僵持**的状态将不会被打破，直至所有哲学家**饿死**。



本节小结



- 1 同步与互斥的概念
- 2 信号量机制的概念
- 3 如何利用信号量解决进程的同步与互斥

阅读教材：47页 ~ 57页